Software for Systematics and Evolution

Systematic Biology (2026), **0**, 0, *pp.* 1–??
doi:10.1093/sysbio/treeflow-diff

TreeFlow: Probabilistic Modelling and Automatic Differentiation for Phylogenetics

Christiaan Swanepoel,[*,1,2]
Mathieu Fourment,[3]
Xiang Ji,[4]
Hassan Nasif,[5]
Marc A Suchard,[6,7,8]
Frederick A Matsen IV,[5,9,10]
Alexei J Drummond[1,11]

[1]Centre for Computational Evolution, The University of Auckland, Auckland, New Zealand;
[2]School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;
[3]Australian Institute for Microbiology and Infection, University of Technology Sydney, Ultimo NSW, Australia;
[4]Department of Mathematics, Tulane University, New Orleans, USA;
[5]Public Health Sciences Division, Fred Hutchinson Cancer Research Center, Seattle, Washington, USA;
[6]Department of Human Genetics, University of California, Los Angeles, USA;
[7]Department of Computational Medicine, University of California, Los Angeles, USA;
[8]Department of Biostatistics, University of California, Los Angeles, USA;
[9]Department of Statistics, University of Washington, Seattle, USA;
[10]Department of Genome Sciences, University of Washington, Seattle, USA;
[11]School of Biological Sciences, The University of Auckland, Auckland, New Zealand, 1010;
Correspondence to be sent to: School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;

Email: cswa648@aucklanduni.ac.nz

Abstract

Probabilistic modelling frameworks are powerful tools for statistical modelling and inference. They are not immediately generalisable to phylogenetic problems due to the particular computational properties of the phylogenetic tree object. TreeFlow is a software library for probabilistic modelling and automatic differentiation with phylogenetic trees. It embeds phylogenetic trees in the TensorFlow Probability framework, and implements

inference algorithms for phylogenetic models given a fixed tree topology. We demonstrate how TreeFlow can be used to quickly implement and assess new models. We also show that it provides reasonable performance for gradient-based inference algorithms compared to specialized computational libraries for phylogenetics.

Key words: phylogenetics; Bayesian inference; probabilistic modelling; variational inference

Introduction

Traditionally, Bayesian phylogenetic analyses have been performed by specialized software ~~(???)~~(???). A number of software packages exist that implement a broad but predefined collection of models and associated specialized inference methods. Typically, inference is handled by carefully crafted but computationally costly ~~stochastic optimisation or~~ Markov chain Monte Carlo (MCMC) methods (**??**). Considerable effort has been invested in improving the computational efficiency of MCMC for phylogenetics ~~(???)~~ (????). In contrast, in other realms of statistical analysis, *probabilistic graphical modelling* or *probabilistic programming* software libraries have entered into widespread use. These allow the specification of almost any model as a probabilistic program, and inference is provided automatically with a generic inference method. Exploiting the power of these tools in phylogenetic analyses could significantly accelerate research by making the process of developing new models and implementing inference faster and more flexible, as evidenced by recent work attempting to reconcile probabilistic modelling and programming frameworks with phylogenetic models (**??**).

Probabilistic modelling tools, such as BUGS (**?**), Stan (**?**), PyMC3 (**?**), and TensorFlow Probability (**?**) allow users to specify probabilistic models by describing the structure of a *probabilistic graphical model* with code. Typically, this involves composing pre-implemented probability distributions by specifying conditional dependencies between

variables. This structure can be used to evaluate a joint probability density function for the model variables. Advancements in automatic inference methods have allowed these tools to perform efficient inference on a wide range of models. Some notable examples, including automatic differentiation variational inference (**?**) (ADVI) and Hamiltonian Monte Carlo (**?**) (HMC), use local gradient information from the model's probability density function to efficiently navigate the parameter space.

Another class of methods for performing statistical inference are *probabilistic programming languages*. These provide more flexibility than graphical modelling methods by explicitly describing the generative process of the model as a *probabilistic program*, rather than just composing probability distributions, and typically include inference schemes that do not require explicit evaluation of a joint probability density. Since the term *probabilistic programming* has been widely adopted to describe probabilistic graphical modelling software, probabilistic programming languages are often qualified as *universal* in that they are not restricted to composing pre-implemented probability distributions. Examples of universal probabilistic programming languages include Church (**?**) and Pyro (**?**).

One key technology that enables gradient-based automatic inference with probabilistic models is *automatic differentiation*. Automatic differentiation refers to methods which efficiently calculate machine-precision gradients of functions, specified by computer programs, without extra analytical derivation or excessive computational overhead compared to the function evaluation. Numerical programs consist of elementary arithmetic operations, and automatic differentiation tools apply the chain rule to these to compute derivatives directly, without requiring the construction of explicit mathematical expressions. Automatic differentiation frameworks that have statistical inference packages built on top of them include Theano (**?**), Pytorch (**?**), JAX (**?**) and TensorFlow (**?**). Some of these extend to non-trivial computational constructs such as control flow and recursion (**?**).

The structure of the phylogenetic tree object is a major barrier to implementing efficient inference for phylogenetic probabilistic models. It is not clear how the association between its discrete and continuous quantities (the topology and branch lengths respectively) should be represented and handled in inference. Also, the combinatorial explosion of the size of the discrete part of the phylogenetic state space presents a major challenge to any inference algorithm. Generic random search methods for discrete variables, as in the naive implementation of MCMC sampling, do not scale appropriately to allow inference on modern datasets with thousands of taxa.

Efforts have already been made to apply probabilistic graphical modelling and probabilistic programming to phylogenetic methods (???)(????). RevBayes (**?**) implements a probabilistic modelling language that is focused on phylogenetic models. However, it requires the user to manually specify an MCMC scheme for performing inference. In other work, it has been shown that universal probabilistic programming languages can be used to express generative processes for trees and generate Sequential Monte Carlo inference schemes for complex speciation models, as well as proposing how they might be used to generate inference schemes for other phylogenetic models, including MCMC samplers for trees (?)(??). Blang (**?**) is a probabilistic modelling framework which supports arbitrary data structures that has been used to implement phylogenetic models and can automatically construct inference schemes. Another tool, LinguaPhylo, provides a domain specific graphical modelling language for phylogenetics and has the capability to automatically generate a specification for an MCMC sampler for inference (?)(?).

Applying gradient-based inference methods to phylogenetics is a major challenge. *Probabilistic path Hamiltonian Monte Carlo* has been developed to use gradient information to sample phylogenetic posteriors across multiple tree topologies (**?**), though moves between tree topologies are not totally informed by gradient information and are similar to the random-walk proposal distributions available to standard MCMC routines. Hamiltonian Monte Carlo proposals for continuous parameters within tree topologies has

been paired with standard MCMC moves between topologies for estimating local mutation rates and divergence times using scalable algorithms for gradient calculation ~~(??)~~(??). Another approach has used Stan to apply automatic differentiation variational inference to phylogenetic inference with a fixed rooted tree topology (**?**). Finally, variational inference has been applied to phylogenetic tree inference using a clade-based distribution on unrooted tree topologies (**?**). This approach has been extended to a more expressive family of approximating distributions (**?**), and applied to rooted phylogenies (**?**).

TreeFlow aims to enable the application of gradient-based inference methods to phylogenetic models. It implements a ~~represention~~ representation of phylogenetic trees in the TensorFlow computational framework which supports automatic differentiation, and a range of associated computations typically used in phylogenetic models of sequence evolution. These are all integrated with the TensorFlow Probability probabilistic modelling framework. TreeFlow provides utilities for applying variational Bayesian inference to phylogenetic models, particularly with a fixed tree topology, and a command line user ~~interace~~interface.

Software Description

TreeFlow is a Python library which provides modular tools for phylogenetic modelling and inference. It ~~also provides~~ provides two primary interfaces: a user interface ~~which can be used for~~ for performing parameter inference in fixed-topology ~~phylogenetic inference with~~ models of a standard form, and a Python developer API for implementing and composing custom phylogenetic models. TreeFlow supports both rooted and unrooted phylogenetic trees, but the user interface is focused on inference with rooted trees. The architecture of the TreeFlow package is displayed in Figure 1.

~~The TreeFlow Python library includes modules for the representation, traversal, and reading and writing of phylogenetic tree . There are modules implementing bijections for phylogenetic trees, phylogenetic posterior approximations, and~~ We first describe the

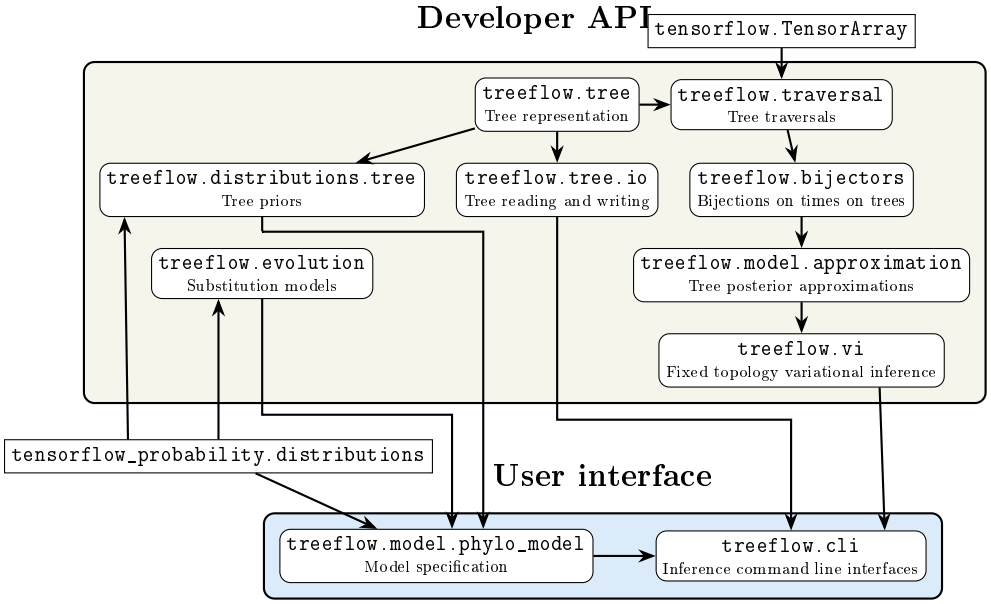


Fig. 1: Diagram showing the architecture of the TreeFlow package. Nodes represent Python modules and the arrows between them denote major dependencies. The dashed rectangles separate the components based on their primary interfaces. Nodes outside the two interface groups denote major external dependencies.

user interface, which provides the most accessible entry point for users wanting to apply TreeFlow to standard phylogenetic analyses. We then describe the developer API, which provides the modular components that underpin the user interface and allow users to implement new models.

User interface

TreeFlow provides command line interfaces for fixed-topology ~~variational Bayesian inference. It also implements tree priors and models of sequence evolution. The user interface implements a model definition formatfor standard phylogenetic models and command line interfaces for variational Bayesian and maximum likelihood~~ inference. These allow inference on standard phylogenetic models such as those performed by specialized software. For inputs, they take a nucleotide sequence alignment in the FASTA or Nexus format, a rooted tree topology in Newick format, and a TreeFlow model specification in

YAML format. Command line interfaces are provided for both automatic differentiation variational inference and maximum a posteriori inference.

~~It is important to note that the primary interface is intended to be the Python developer API, which~~ Because many phylogenetic analyses use models of a similar structure, TreeFlow provides a format for concisely specifying this kind of model. A single unpartitioned multiple sequence alignment can be modelled with a single substitution model, tree prior, and associated parameter priors. The model specification is a nested dictionary, with keys at various levels corresponding to model components, their parameters, and distributions used as priors. This is transformed into a TensorFlow Probability joint distribution. The model definition structure can be serialized in the YAML markup format which provides a text-based model definition format. An example of this is shown in Supplementary Appendix Figure S1. The substitution and tree models can be drawn from the models described below, as well as parameter priors from a range of standard probability distributions implemented in TensorFlow Probability. This model definition format is not intended to cover the full range of probabilistic models that can be implemented with TreeFlow's Python API, but rather provide a convenient input for TreeFlow's command line interfaces.

Developer API

The developer API allows users to implement new models and compose them with existing model components in a flexible structure, while leveraging the inferential machinery provided by TreeFlow. The ~~user interface is provided to simplify testing and demonstration of TreeFlow, while also potentially making TreeFlow's scalable inference methods more accessible~~TreeFlow Python library includes modules for the representation, traversal, and reading and writing of phylogenetic trees. There are modules implementing bijections for phylogenetic trees, phylogenetic posterior approximations, and fixed-topology variational Bayesian inference. It also implements tree priors and models of sequence

evolution. TensorFlow Probability distributions can be composed into a *probabilistic graphical model* using TensorFlow Probability's joint distribution functionality (?). The code to specify a joint distribution provides a concise representation of the model used in a data analysis. The ability to implement phylogenetic models in this framework means that automatic inference algorithms implemented in TensorFlow can be leveraged, as well as other utilities such as sampling from the joint prior. The subsections below describe the major components of the developer API.

Tree representation

TreeFlow is built on TensorFlow, a computational framework for machine learning. TreeFlow leverages TensorFlow's capabilities for accelerated numerical computation and automatic differentiation. TensorFlow operations can be executed on either the CPU or GPU, and as a result TreeFlow can make use of either type of processor. Many of TensorFlow's CPU operations are parallelized, allowing TreeFlow to make use of multiple processor cores. All of the benchmarks and examples in this paper, and TreeFlow's command line interfaces, use the CPU for computation.

TensorFlow's core computational object is the Tensor, a multi-dimensional array with a uniform data type. Phylogenetic trees are not immediately at home in a Tensor-centric universe, as they are a complex data structure, often defined recursively, with both continuous and discrete components. TreeFlow represents trees as a structure of Tensors. This structure contains floating point Tensors representing the times and branch lengths, and integer Tensors representing the topology, all encapsulated in a Python object. The topological Tensors include indices of node parents, children, and for pre- and post-order traversals. TensorFlow has extensive support for structures of Tensors such as the TreeFlow tree class, including using them as arguments to compiled functions and defining probability distributions over complex objects. This means computations and models involving phylogenetic trees can be expressed naturally.

Tree distributions

TreeFlow uses the existing probabilistic modelling infrastructure provided by TensorFlow Probability, which implements standard statistical distributions and inferential machinery (?). The Distribution interface encapsulates functionality for sampling from a distribution and computing its probability density or mass function. TreeFlow implements a tree distribution base class which allows TensorFlow Probability Distributions to be defined on TreeFlow's phylogenetic tree representation.

TreeFlow implements Distributions for some commonly-used generative models for phylogenetic trees. The models currently implemented, Kingman's coalescent (?) and the Birth-Death-Sampling speciation process (?), can be used as *tree priors* in phylogenetic ~~probabilsitic~~ probabilistic models and to infer parameters related to population dynamics from genetic sequence data. Since sampling from these distributions is non-trivial and not required for variational inference, only the probability densities of these models are currently implemented.

Models of sequence evolution

The models of nucleotide sequence evolution currently implemented in TreeFlow are the Jukes-Cantor (?), HKY85 (?), and General Time Reversible (GTR) (?) models. TreeFlow includes a standard approach for dealing with heterogeneity in mutation rate across sites based on ~~a~~ marginalizing over a discretized site rate distribution (?). The probabilistic modelling framework, however, allows for the use of any appropriate distribution as the base site rate distribution rather than just the standard single-parameter Gamma distribution. For example, it is straightforward to replace the base Gamma distribution with a Weibull distribution, which has a quantile function that is much easier to compute (?). Thanks to TensorFlow's vectorized arithmetic operations, it is also natural to model variations in mutation rate over lineages by specifying parameters for multiple rates (possibly with a hierarchical prior) and multiplying by the branch lengths of

the phylogenetic tree. Models which can be naturally expressed this way include the log-Normal random local clock (**?**), which is implemented with TreeFlow's model definition format, and auto-correlated relaxed clock models (**?**), which would be straightforward to implement with the traversal module in the Python API.

Tree traversals

To perform model computations involving the phylogenetic tree, it is necessary to traverse the tree according to the structure specified by its topology. Since this structure is less accessible in an array-based tree representation, and native Python control flow cannot be used when the topology of the tree is a dynamic variable, TreeFlow provides utilities to perform tree traversals. These are used to efficiently implement some core computations and also made accessible to developers for extending TreeFlow's functionality.

When iterating over the nodes of the tree in an automatic differentiation framework it is important to consider computational complexity. Since modern genomic datasets can result in the number of nodes in the tree being extremely large, it is crucial to maintain linear scaling with the size of the tree. Any computation that produces values for every node and requires access to results for multiple previous nodes in the traversal might incur a quadratic computational cost. This could occur when the topology is a dynamic variable as, at a given step of the iteration, values from any element of the Tensor representation might ~~be~~ need to be accessed. Since TensorFlow's automatic differentiation framework depends on the immutability of Tensors, an intermediate representation of the state for the whole tree needs to be maintained for every step of the iteration, leading to a quadratic complexity.

TreeFlow's solution to this scaling issue is to implement tree traversals using TensorFlow's `TensorArray` construct (**?**). The TensorArray is a data structure representing a collection of tensors. The TensorArray relaxes the requirement of immutability to a *write-once* property enforced on its constituent tensors. The

TensorArray tracks dependencies between values so automatic differentiation can appropriately propagate gradients back through computation in linear time.

A notable computation that makes use of the tree traversal framework described above is the *phylogenetic likelihood*, the probability of a sequence alignment given a phylogeny and model of sequence evolution. This typically involves integrating out character states at unsampled ancestral nodes using a *dynamic programming* computation known as *Felsenstein's pruning algorithm* (**?**). Considerable attention has been given to the problem of efficiently computing gradients of branch lengths with respect to the phylogenetic likelihood (**?**). The dynamic programming algorithm is implemented as a TensorArray-based *postorder* traversal. The branch-length gradient computed through automatic differentiation demonstrates linear scaling with respect to the number of taxa in the tree in our benchmarks (see Benchmarks section below). Our implementation also allows computation of gradients which respect to any substitution model parameters.

Another useful tool for phylogenetic inference implemented in TreeFlow is the *node height ratio transform* (**?**). This has been used to infer times on phylogenetic trees using maximum likelihood in the PAML software package (**?**). The ratio transform parametrizes the internal node heights of a tree as the ratio between a node's height and its parent's height. This has been applied to Bayesian phylogenetic inference in the context of automatic differentiation variational inference (**?**) and Hamiltonian Monte Carlo ~~(?)~~(**?**). The computation of node heights from ratios is implemented in TreeFlow as a TensorArray-based *preorder* traversal.

Bijectors on phylogenetic trees

The ratio transform described above is wrapped into a TensorFlow Probability Bijector which provides an interface for transforming real-valued distributions into phylogenetic tree distributions. The Bijector is an interface which represents a transformation of samples from a multivariate probability distribution. It requires

implementation of the forward transformation and its inverse (which necessarily exists if the transformation is a bijection), as well as the determinant of the Jacobian matrix of the transformation. This can be used by TensorFlow Probability to perform change of variable, rewriting the probability density of the base distribution in the space the bijector transforms it to.

The ratio transformation has a triangular Jacobian matrix, which means computing the determinant required for the change of variable formula can be computed in linear time with respect to the number of internal node heights (**?**). Using change of variable, a multivariate distribution on ratios can be transformed into a distribution on the internal node heights of a rooted phylogeny. Other TensorFlow Probability Bijectors can be used to transform real-valued distributions into ratios. Another Bijector implemented in TreeFlow ~~is the composition of~~ combines node heights with a fixed tree topology to ~~form~~ produce a complete phylogenetic tree object. These bijectors can be ~~chained to transform a real-valued distribution into a fixed-topology distribution over TreeFlow's tree objects~~composed sequentially — for example, transforming unconstrained real values to ratios, then to node heights, then to a full tree — yielding a probability distribution over branch lengths for a given topology. The result can be used as an approximation to a phylogenetic posterior distribution and combined with models such as TreeFlow's tree priors and phylogenetic likelihood.

Variational inference

One form of statistical inference for which gradient computation is essential is variational Bayesian inference (**?**). The goal of Bayesian inference is to characterise a *posterior distribution* which represents uncertainty over model parameters. Variational Bayesian inference achieves this by optimizing an approximation to the posterior distribution which has more convenient analytical properties. Typically, the optimisation routine used in variational inference can scale to a larger number of model parameters than

the random-walk sampling methods used by MCMC methods.

One concrete implementation of variational inference is *automatic differentiation variational inference* (ADVI) (**?**). ADVI can perform inference on a wide range of probabilistic graphical models composed of continuous variables. It automatically constructs an approximation to the posterior by transforming a convenient base distribution to respect the domains of the model's component distributions. It then optimizes this approximation using stochastic gradient methods (**??**). Typically, independent Normal distributions are used as the base distribution for computational convenience. This is known as the *mean field approximation*, and for posterior distributions that have significant correlation-structure, skew, multi-modality, or heavy tails, can introduce error in parameter estimation (**?**). Possible solutions to this approximation error include highly flexible variational approximations with large numbers of parameters (**?**) or structured approximations that are informed by the form of the model (**?**).

TreeFlow leverages the stochastic gradient optimizers already implemented in TensorFlow and the variational inference objectives in TensorFlow Probability. The discrete topology element of phylogenetic trees is an obstacle in the usage of these algorithms, which are typically restricted to continuous latent variables. Often, the phylogenetic tree topology is not the latent variable of interest, and is not a significant source of uncertainty (**?**). This can be the case when divergence times or other substitution or speciation model parameters are the focus. In this scenario, useful results can be obtained by performing inference with a fixed tree topology, such as one obtained from fast maximum likelihood methods. This is the approach taken by the NextStrain platform (**?**), which uses the scalability afforded by a fixed tree topology to allow large-scale rapid phylodynamic analysis of pathogen molecular sequence data. Using a variational approximation with a fixed topology in TreeFlow makes the application of ADVI to phylogenetic models straightforward. When using variational inference in TreeFlow, care must be taken if it is possible the tree topology for a given dataset is difficult to infer by

assessing how sensitive the inference results are to it.

Although TreeFlow's current variational inference scheme is limited to a fixed tree topology, the rest of the library is not constrained in this manner. The Tensor-based tree representation means that the topology is part of TensorFlow's computational graph. Additionally, computations such as the phylogenetic likelihood are implemented with support for a dynamic tree topology. This means that ~~gradient-based inference schemes that also infer the tree topology, such as the~~ existing approaches to variational inference ~~for phylogenetic trees (?) , could naturally be implemented in TensorFlow with TreeFlowproviding~~ over tree topologies (?) could be implemented using TreeFlow's model specification and likelihood evaluation~~functionality~~, extending the library to support topology inference in the future.

Model approximations

TreeFlow implements posterior approximations for ADVI using TensorFlow Probability's bijector framework to transform a base distribution. Approximations are constructed automatically from probabilistic models in the form of TensorFlow Probability joint distribution objects. A base distribution, by default an uncorrelated Normal, is split into the variables of the joint distribution, and transformed to match the support of the model variables. The base distribution for the phylogenetic tree is transformed using a chain of bijectors. Learnable parameters to optimize are introduced by adding an initial trainable transformation to the base distribution. For the mean field approximation, this is an affine elementwise shifting and scaling operation.

ADVI opens the door to using TensorFlow's neural network framework to implement deep-learning-based variants such as variational inference with *normalizing flows* (?), which transform the base distribution through invertible trainable neural network layers to better approximate complex posterior distributions. Approximations based on *inverse autoregressive flows* (?) are implemented in TreeFlow, but since their

computational and memory requirements scale quadratically with the number of model variables they are not ideal for large phylogenetic analyses.

Model definition

TensorFlow Probability distributions can be composed into a *probabilistic graphical model* using TensorFlow Probability's joint distribution functionality (?). The code to specify a joint distribution provides a concise representation of the model used in a data analysis. The ability to implement phylogenetic models in this framework means that automatic inference algorithms implemented in TensorFlow can be leveraged, as well as other utilities such as sampling from the joint prior.

Because many phylogenetic analyses use models of a similar structure, TreeFlow provides a format for concisely specifying this kind of model. A single unpartitioned multiple sequence alignment can be modelled with a single substitution model, tree prior, and associated parameter priors. The model specification is a nested dictionary, with keys at various levels corresponding to model components, their parameters, and distributions used as priors. This is transformed into a TensorFlow Probability joint distribution. The model definition structure can be serialized in the YAML markup format which provides a text-based model definition format. An example of this is shown in Figure ??. The substitution and tree models can be drawn from those described above, as well as parameter priors from a range of standard probability distributions implemented in TensorFlow Probability. This model definition format is not intended to cover the full range of probabilistic models that can be implemented with TreeFlow's Python API, but rather provide a convenient input for TreeFlow's command line interfaces.

Command line interfaces

As well as a library for probabilistic modelling with TensorFlow Probability, TreeFlow provides command line interfaces for fixed-topology inference. These allow

~~inference on standard phylogenetic models such as those performed by specialized software. For inputs, they take a nucleotide sequence alignment in the FASTA or Nexus format, a rooted tree topology in Newick format, and a TreeFlow model specification in YAML format. Command line interfaces are provided for both automatic differentiation variational inference and maximum a posteriori inference.~~

Biological examples

We used TreeFlow to perform fixed-topology phylogenetic analyses of two datasets. The first is an alignment of 62 mitochondrial DNA sequences from carnivores (**?**). The second is a dataset of 980 influenza sequences (**?**). In both datasets maximum likelihood unrooted topologies are estimated using RAxML (**?**). These topologies are rooted with Least Squares Dating (**?**).

Probabilistic modelling and model selection

In the carnivores dataset, we demonstrate the flexibility of probabilistic modelling with TreeFlow by investigating variation in the ratio of transitions to transversions in the nucleotide substitution process across lineages in the tree. Early maximum likelihood analyses of mitochondrial DNA in primates detected variation in this ratio, but without a biological basis, it was attributed to a saturation effect (**?**). A later simulation-based investigation showed that this was a reasonable explanation, which exposed the limitations of nucleotide substitution models for estimating the length of branches deep in the tree (**?**).

This problem could be approached by means of Bayesian model comparison; a lack of preference for a model allowing between-lineage variation of the ratio could indicate that the substitution model lacks the power to separate variation 'signal' from saturation 'noise'. Firstly, we construct a standard phylogenetic model with a HKY substitution model with a single transition-transversion ratio parameter (*kappa*). We perform inference using ADVI, and also using MCMC as implemented in BEAST 2 ~~(?)~~(v2.7.7; (**?**)). Since

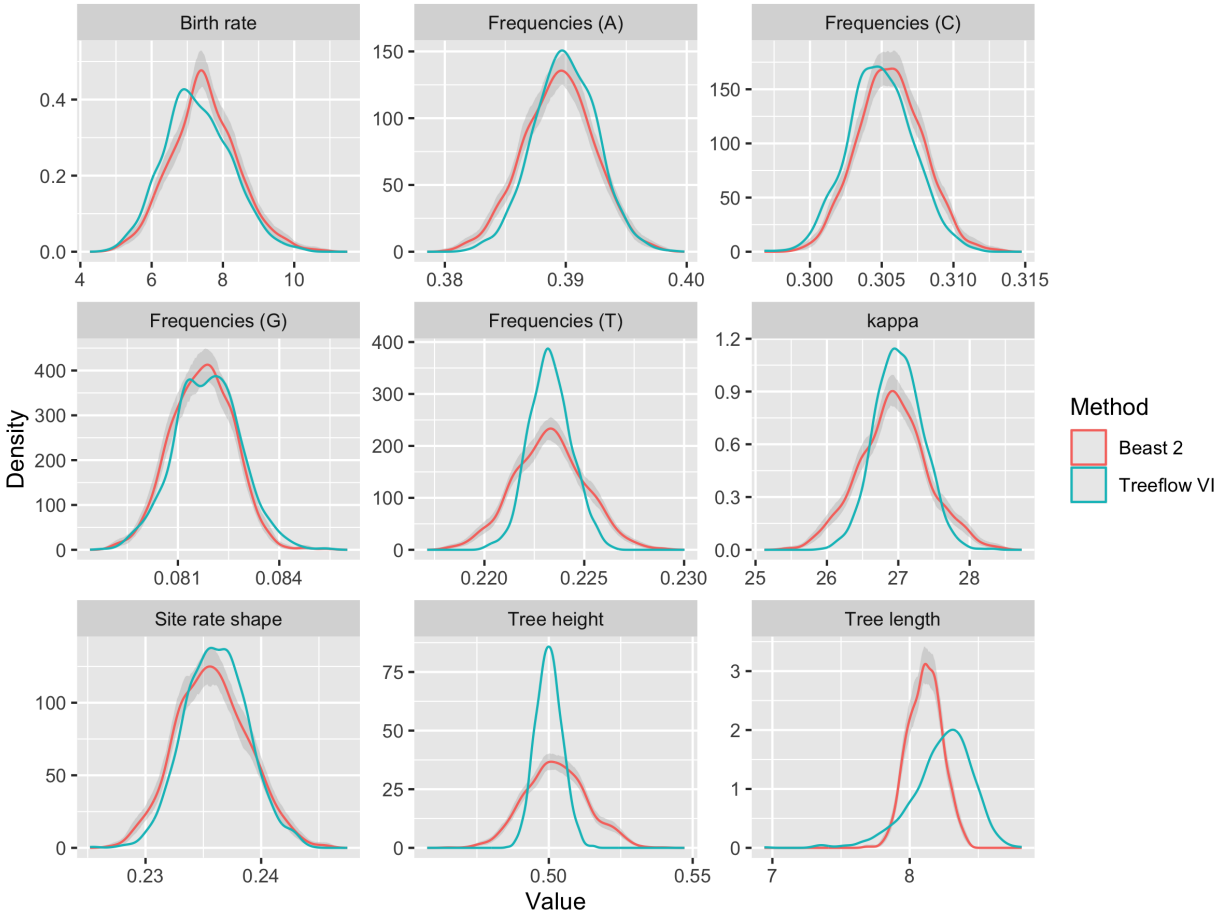
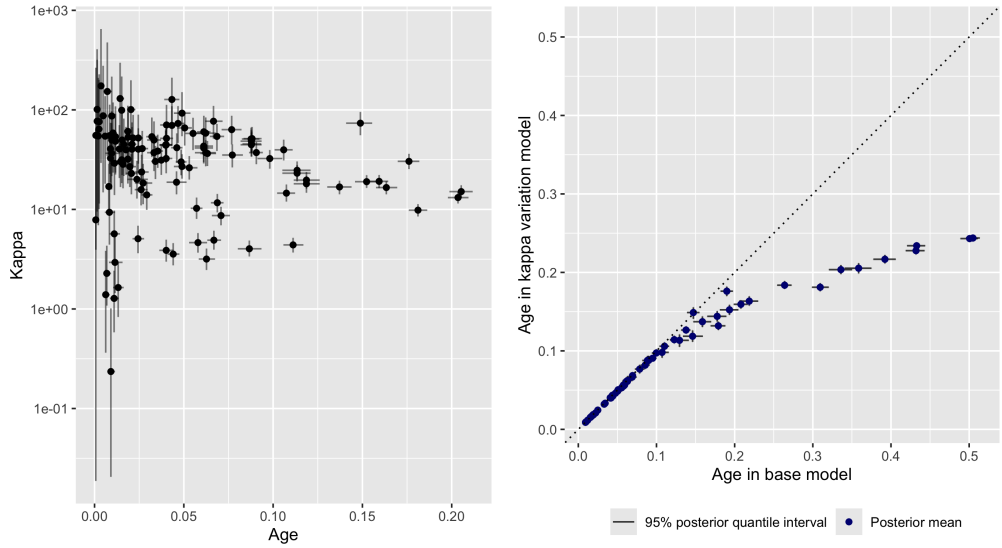


Fig. 2: Carnivores base model posterior marginal parameter estimates

this model is of the form of a standard phylogenetic analysis, it could be fit using TreeFlow's command line interface. Figure 2 compares the marginal parameter estimates obtained from TreeFlow and BEAST 2. The error in variables with larger discrepancies in distribution, apparent in the estimates of the frequencies and tree height, can be attributed to the approximation error introduced by ADVI. Most importantly, the estimate of the parameter of interest, kappa, appears reasonable.

We then altered this model to estimate a separate ratio for every lineage on the tree; the implementation of this was simple and scalable as a result of TensorFlow's vectorization and broadcasting functionality. ~~Figure ?? shows the~~ The model definition code for the base model ~~,~~ and the small changes required to model variation in the kappa



(a) Lineage age (number of expected substitutions per site before the present) vs estimated kappa for carnivores dataset

(b) Node age estimates for base model vs per-lineage kappa variation model

Fig. 3: Parameter estimates for per-lineage transition/tranversion ratio (kappa) variation model on carnivores dataset

parameter across lineages are shown in Supplementary Appendix Figure S2. We compared the models using estimates of the marginal likelihood (**?**). The *marginal likelihood*, or *evidence*, the integral of the likelihood of the data over model parameters, is typically analytically intractable and challenging to compute using MCMC methods (**?**). The closed-form approximation to the posterior distribution provided by variational inference ~~means we could estimate the marginal likelihood using importance sampling~~ can be used as a proposal distribution for importance sampling (**?**), a utility provided by TreeFlow. ~~This corrects for some of the posterior approximation error described above. The log marginal likelihood for the base model was -193606.4 and for the model with kappa variation it was -192414.4~~Because the importance weights account for the discrepancy between the variational approximation and the true posterior, this produces more accurate marginal likelihood estimates than the variational bound alone.

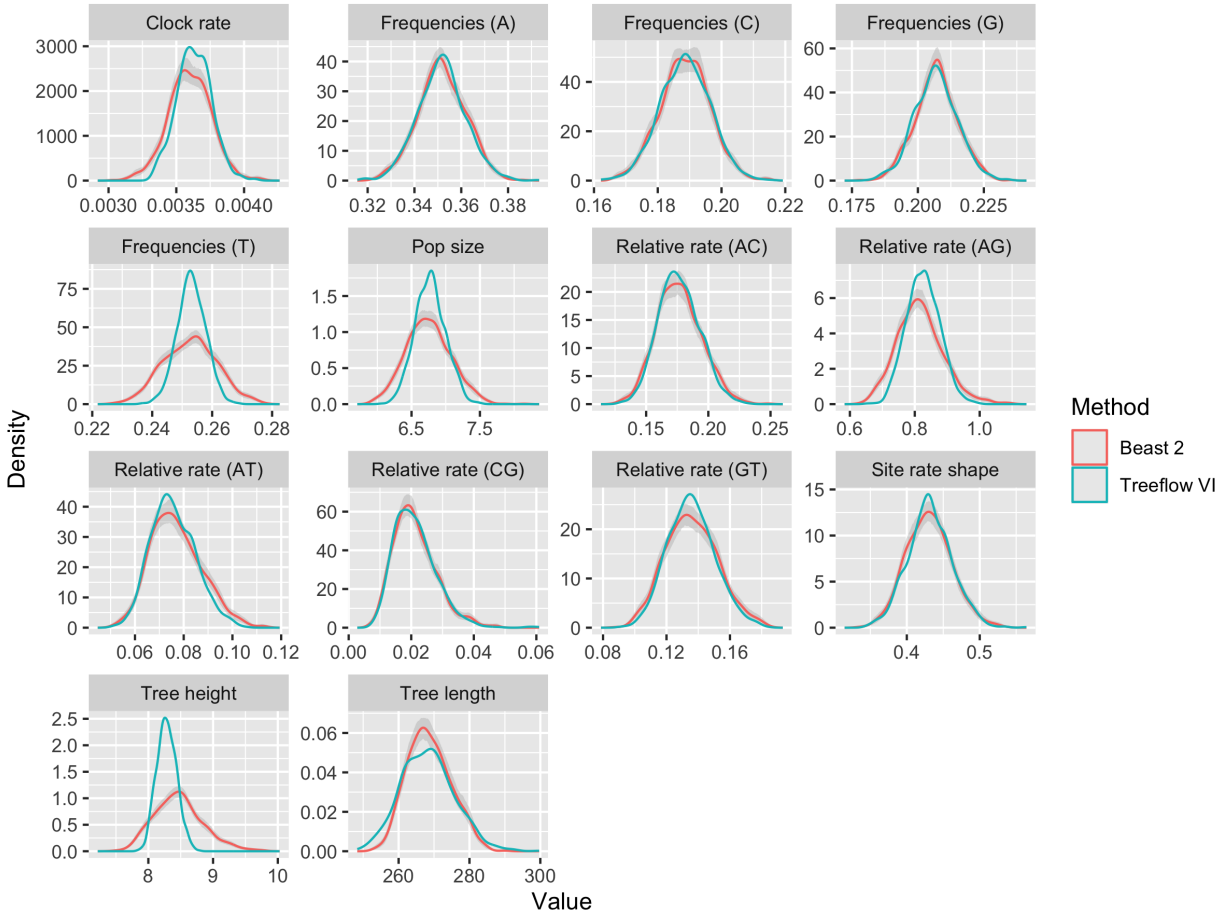
The estimated per-lineage kappa parameters are shown in Figure 3a. Kappa estimates decline with increasing age of the lineage, which agrees with the results of the

previous studies. The estimated log marginal likelihood for the base model was -193607.0 and for the model with kappa variation it was -192415.1, indicating that the model with transition-transversion ratio variation across lineages ~~was higher than for the base model~~is preferred. This means that, under the other components of this model, the data supports variation in the ratio. This does not necessarily imply that transition-transversion ratio in the true generative process of the data varies in the same way, but could be a useful consideration in designing more sophisticated models of nucleotide substitution. The growth in uncertainty in kappa estimates deeper in the tree supports previous conclusions that nucleotide substitution models are unable to effectively estimate the number of substitutions on older branches (**?**). Figure 3b shows that the kappa variation model shortens older branches, leading to a substantially reduced overall tree height estimate, with proportionally similar uncertainty in node height estimates. This is not a proper dating analysis because it does not account for uncertainty in the mutation rate and lacks time calibration. However, it is evident that the inclusion or exclusion of heterogeneity in the transition-transversion ratio affects the time estimates.

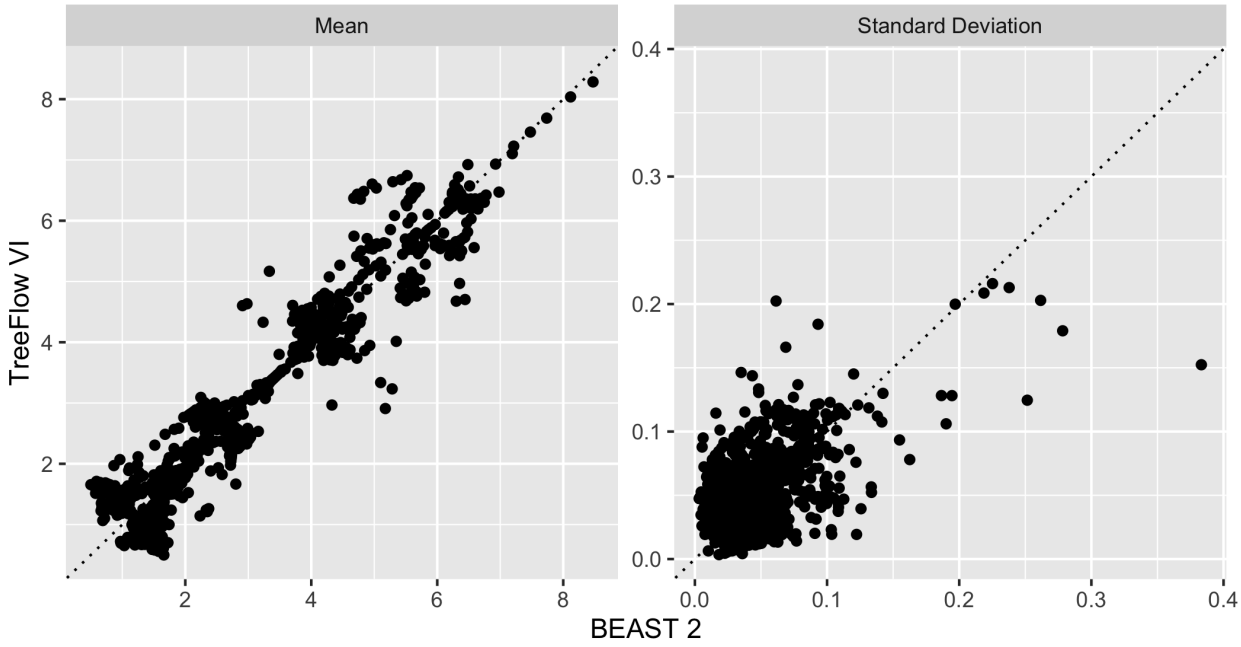
~~YAML model definition for the analysis of the influenza dataset~~ This example demonstrates the power of TreeFlow's probabilistic modelling approach: extending the standard model to include per-lineage kappa variation required only a few lines of code changes (Supplementary Appendix Figure S2), and the variational inference and marginal likelihood estimation machinery could be applied to the new model without any additional implementation effort.

Scalable inference

In the analysis of the 980-taxon influenza dataset, we demonstrate the scalability of variational inference. We performed variational inference using TreeFlow's command line interface. The model used for inference included a coalescent prior with a constant population size on the tree, a strict molecular clock, a discretized Gamma model of site



(a) Parameter marginal posterior distributions



(b) Node height posterior distribution statistics

Fig. 4: Parameter estimates from analysis of the influenza dataset

rate variation and a GTR substitution model. The YAML model definition code for this model, including parameter priors, is shown in ~~Figure ??~~Supplementary Appendix Figure S1. This parameterization of the GTR substitution model (here named `gtr_rel`), with independent priors on five of the relative rates and one held fixed to 1, is used to allow comparison of parameter estimates with BEAST 2. It is also possible to use a six-rate GTR parameterisation with a Dirichlet prior in TreeFlow such as used in other phylogenetics software (**??**).

Figure 4a shows the marginal parameter estimates obtained from this dataset, compared to those obtained from a fixed-topology MCMC analysis using BEAST 2. The posterior distributions of most parameters are approximated with high fidelity by the mean field approximation. The uncertainty in the tree height, coalescent effective population size, and clock rate are slightly underestimated. This is likely a result of using an approximation that ignores the correlations between these parameters that are present in the posterior. The objective function used in ADVI, the KL divergence between the approximating and posterior distributions, heavily penalises regions of the approximation where there is no posterior support, which typically results in low uncertainty estimates.

Figure 4b compares the divergence time estimates. TreeFlow's mean field variational approximation assumes the posterior distribution on the times of the tree are independent Normal distributions transformed through the node height ratio transformation to respect the time constraints of the tree. However, the true posterior is almost certainly not of this form, so the approximation introduces error. In general, the mean of the posterior distribution is well approximated, in particular for the oldest seven nodes of the tree. The posterior means of other divergence times are generally close but not identical to the true posterior. The error is more apparent in the posterior uncertainty; the standard deviation estimates produced by variational inference often differ substantially from the posterior estimated by BEAST 2. While mean field variational inference seems effective for estimating the parameters of the phylogenetic model the

estimates of divergence times appear less reliable. Variational approximations that better capture the form of the true posterior would improve the quality of these estimates.

BEAST 2 MCMC sampling was run for 30,000,000 iterations using the standard single-threaded MCMC routine. Convergence was checked using *effective sample size* (ESS), which was computed for all parameters using ArviZ (**?**). ~~Multiple~~ BEAST 2's built-in operator autotuning was used, and multiple preliminary runs were performed to ~~tune MCMC~~ manually adjust operator weights to improve ESSs. Generally, the minimum acceptable ESS is 100, though 200 is preferred (**?**). The analysis resulted in a minimum effective sample size of ~~125.~~ 103. The wall clock runtime for the BEAST 2 analysis was ~~5 hours and 57~~ 6 hours and 21 minutes. Convergence for the TreeFlow-based variational inference analysis was ~~checked visually by examining parameter traces~~ assessed by monitoring the evidence lower bound (ELBO) over the optimization routine and visually examining parameter traces. The TreeFlow analysis converged in ~~20~~22,000 iterations which took ~~3 hours and 3~~ 10 hours and 30 minutes. This ~~shows~~ demonstrates that variational inference with TreeFlow is a ~~reasonable~~ viable option for large phylogenetic datasets, providing substantial speed improvements over MCMC despite the more expensive gradient computations required~~, and that~~ . TreeFlow's ~~slower~~ phylogenetic likelihood implementation performs well enough to be useful in real-world inference scenarios~~where the sacrifice in accuracy from the variational approximation is permissible~~, and the entire analysis was specified using TreeFlow's YAML model definition format (Supplementary Appendix Figure S1) without any custom code.

Benchmarks

The phylogenetic likelihood is the computation that dominates the computational cost of model-based phylogenetic inference. We benchmark the performance of our TensorFlow-based likelihood implementation against specialized libraries. Since gradient computations are of equal importance to likelihood evaluations in modern automatic

inference regimes, we also benchmark computation of derivatives of the phylogenetic likelihood, with respect to both the continuous elements of the tree and the parameters of the substitution model. A clear difference emerges in the implementation of derivatives; TreeFlow's are based on automatic differentiation while bespoke libraries need analytically-derived gradients. Therefore, we do not necessarily expect our implementation to be as fast as bespoke software, but it does not rely on analytical derivation of gradient expressions for every model and therefore automatically supports a wider range of models.

We compare the performance of TreeFlow's likelihood implementation against BEAGLE (**?**). BEAGLE is a library for high-performance computation of phylogenetic likelihoods. Recent versions of BEAGLE implement analytical gradients with respect to tree branch lengths (**?**). We use BEAGLE via the `bito` software package (**?**), which provides a Python interface to BEAGLE's branch length gradients and also numerically computes derivatives with respect to substitution model parameters using a finite differences approximation. This hybrid approach has been shown to perform as well as fully analytical substitution model parameter gradients on large datasets (**?**).

We also compare TreeFlow with a simple likelihood implementation (**?**) based on another automatic differentiation framework, JAX (**?**). JAX uses an eager execution model where execution of an operation immediately results in the computation being performed. In contrast, TensorFlow's function mode, which is used by TreeFlow in the benchmarks, constructs a graph representation of the computation which is processed and can then be executed. As a result the phylogenetic likelihood in JAX can be implemented with native Python control flow even with a dynamic tree topology.

Benchmarks are performed on simulated data. Simulation allows the generation of a large number of data replicates with appropriate properties. Since we want to investigate the scaling of likelihood and gradient calculations with respect to the number of sequences, we simulate data for a range of sequence counts. Sequence counts are selected as increasing powers of 2 to better display asymptotic properties of the implementations. We simulate

data with sequence counts ranging from 32 to 2048. Trees are simulated under a coalescent model for a given number of taxa, and then nucleotide sequences of length 1000 are simulated under a fixed rate of evolution and a HKY substitution model. Tree and sequence simulations are performed using BEAST 2 (**?**).

We benchmark ~~4~~ four distinct computations on these datasets. Firstly, for each replicate we compute the phylogenetic likelihood under a very simple model of sequence evolution. This uses a Jukes-Cantor model of nucleotide substitution and no model of site rate variation. Secondly, we calculate derivatives with respect to branch lengths under this simple model. Thirdly, we compute the likelihood under a more sophisticated substitution model, with a discretized Weibull distribution with ~~4~~ four rate categories to model site rate variation and a GTR model of nucleotide substitution. This Weibull distribution is used for site rate variation in `bito` instead of the widely used Gamma distribution because its inverse cumulative distribution function (required for computing rates for the discretized categories) has a simpler form. Because this is a minor part of the computation, only performed once for each rate category per likelihood evaluation, TreeFlow's scaling should be unaffected if the standard Gamma distribution is used. Finally, we compute derivatives with respect to both branch lengths and the parameters of the substitution model (the ~~6~~ six GTR relative substitution rates, ~~4~~ four base nucleotide frequencies, and the Weibull site rate shape parameter). Each computation is performed 100 times on 10 simulated datasets.

Figure 5 shows the results of the benchmarks with a log scale on both axes. For likelihood computation with both models and branch gradient computation with the simple model `bito`/BEAGLE are at least an order of magnitude faster than TreeFlow. This is expected as BEAGLE is a highly optimized special-purpose library written in native code. The performance gap grows much smaller for computing the gradients of the more complex substitution model. `bito` performs at least 2 likelihood evaluations for each additional parameter when calculating the gradient with respect to substitution model parameters, while the overhead for substitution model parameters with automatic

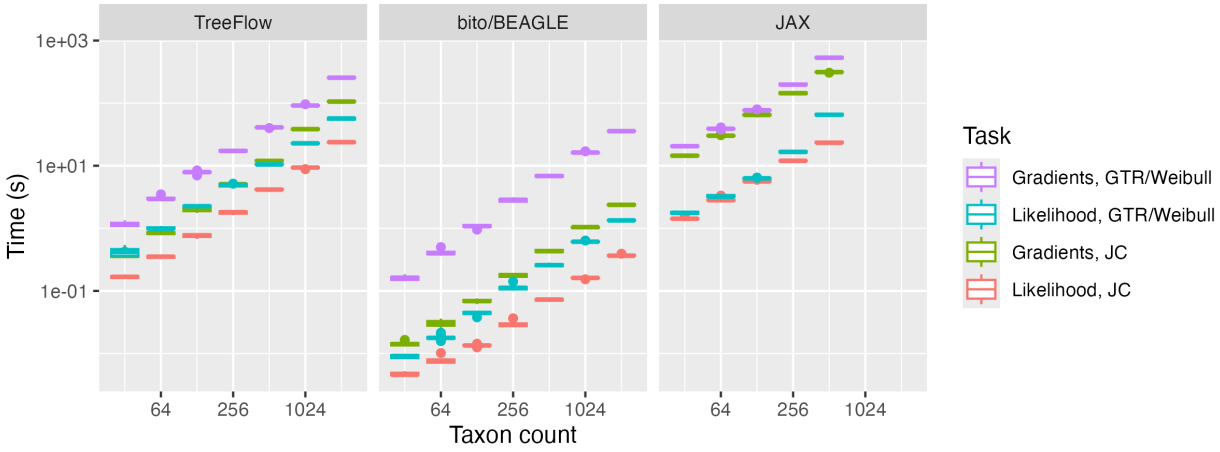


Fig. 5: Times from phylogenetic likelihood benchmark (100 evaluations for 1000 sites)

Model	Computation	Method	Mean time (512 taxa)	log-log slope
JC	Likelihood	TreeFlow	~~6.23~~ 4.16	~~1.13~~ 1.19
		bito/BEAGLE	~~0.16~~ 0.07	~~1.27~~ 1.07
		JAX	~~168.79~~ 23.65	~~1.15~~ 1.01
	Gradients	TreeFlow	~~14.55~~ 11.98	~~1.23~~ 1.36
		bito/BEAGLE	~~0.87~~ 0.43	~~1.33~~ 1.25
		JAX	~~644.22~~ 312.68	~~1.19~~ 1.11
GTR/Weibull	Likelihood	TreeFlow	~~10.73~~ 10.52	~~1.12~~ 1.16
		bito/BEAGLE	~~0.58~~ 0.26	~~1.40~~ 1.23
		JAX	~~908.24~~ 65.39	~~1.58~~ 1.27
	Gradients	TreeFlow	**~~36.14~~ 41.00**	**~~1.34~~ 1.27**
		bito/BEAGLE	**~~14.74~~ 6.84**	**~~1.41~~ 1.31**
		JAX	**~~2801.35~~ 533.91**	**~~1.58~~ 1.17**

Table 1: Results of phylogenetic likelihood benchmark. Times for gradients for the GTR/Weibull are highlighted as they are the most relevant computation for gradient-based inference on real data.

differentiation is minimal. As a result, it is likely that TreeFlow would be a reasonable choice compared to `bito`/BEAGLE for substitution models with even more parameters (e.g. amino acid substitution models (**?**)), or those where the number of parameters grows with the number of sequences, as in the example using the carnivore dataset above.

For typical real-world phylogenetic analyses, a model with many parameters, such as the GTR/Weibull model in our benchmarks, is used. For modern Bayesian inference

methods such as variational inference and Hamiltonian Monte Carlo, the gradient is the primary computation performed at each iteration. We observe that TreeFlow's times for this combination of model and computation are within an order of magnitude of `bito`/BEAGLE's, around 2-3 times slower. Therefore for applied analyses the automatic differentiation-based computations of TreeFlow presents a reasonable alternative to specialized software while simultaneously offering greater flexibility.

We also observe that the runtimes for TreeFlow are roughly an order of magnitude less than those of the JAX-based implementation. These indicate that the control flow constructs and execution model of TensorFlow are a good choice for implementing tree-based computations compared to the eager execution model of JAX.

Table 1 shows the coefficients obtained from fitting a linear model to the benchmark times where the predictor is the log-transformed number of sequences and the target is the log-transformed runtime. The slope parameter estimates from these fits is a rough empirical estimate of the polynomial degree of the computational scaling. The slope parameters for all TreeFlow computations are well below 2 and indicate a roughly linear scaling with the size of the data. For the JC benchmark, the only gradients computed are with respect to branch lengths, so the whole computation for `bito`/BEAGLE is the analytical linear-time branch gradient. TreeFlow's scaling is certainly not worse in this case, indicating that the TensorArray-based likelihood implementation enables linear-time branch gradients.

Discussion

The probabilistic modelling approach enabled by the TreeFlow Python API in conjunction with TensorFlow Probability has similar goals to other phylogenetic probabilistic modelling software. Two notable examples, RevBayes (?) and Blang (?) provide a generic model definition and inference engine which also support phylogenetic models and tree inference. Some notable differences exist between TreeFlow and these which may prove advantageous for certain users. Firstly, RevBayes requires the user to

manually specify an MCMC operator scheme, including parameters and weights. This may require substantial user time and expertise to achieve correct and efficient inference. Blang and TreeFlow, on the other hand, are focused on automatic inference methods that require less user input. Secondly, TreeFlow and its Python model specification API are implemented in a widely used computational framework (TensorFlow) and programming language (Python). As a result it can potentially leverage other computational tools implemented in TensorFlow, including less common distributions, machine learning models, differential equation solvers and alternative inference schemes. The automatic differentiation capabilities TensorFlow affords mean TreeFlow can interface with the large and growing collection of models that rely on gradient-based inference. On the other hand, the choice of computational framework incurs a performance cost as evidenced by the results of benchmarking. Blang models can make use of external Java code, but the ecosystem of related software in Java is less developed, and RevBayes does not provide any functionality for interfacing with software that is not implemented in the Rev language.

The combination of flexibility and scalability of libraries like TreeFlow has the potential to be useful for rapid analysis of large modern genetic datasets, such as those assembled for tracking infectious diseases (**?**). For this purpose, TreeFlow's true power would be unlocked with the implementation of improved modelling and inference tools for evolutionary rates and population dynamics.

For principled dating analyses, uncertainty in evolutionary rate parameters must be considered. Posterior distributions involving these parameters may have significant correlation structure which would be ignored by variational inference using a mean-field posterior approximation. These parameters typically receive special treatment in MCMC sampling routines (**??**). As observed in our analysis on the influenza dataset, the mean field approximation fails to accurately capture the posterior distribution on all internal divergence time estimates. Implementing structured posterior approximations for these models could substantially improve the reliability of parameter estimates obtained using

variational inference while scaling to extremely large datasets. For example, the known correlation between the clock rate and tree height could be captured by using a block-diagonal covariance structure in the variational family, or by applying normalizing flows (?) to model more complex dependencies.

Additionally, implementing flexible models for population dynamics, such as nonparametric coalescent models (???), could provide valuable insights from large datasets. These models would result in complex posterior distributions, which would need to be accounted for in a variational inference scheme. If these coalescent models could be made to work effectively in TreeFlow, the probabilistic graphical modelling paradigm would allow for rapid computational experimentation with novel models of time-varying population parameters.

Finally, the most significant functionality for phylogenetic inference missing from TreeFlow is inference of the tree topology. TreeFlow's tree representation could allow for the topology to become an estimated parameter, such as in existing work on variational Bayesian phylogenetic inference (?). Efficiently implementing the computations required for these algorithms in the TensorFlow framework would ~~certainly~~ be a major computational challenge, but would be supported by TreeFlow's representation of the tree topology as a dynamic variable inside the computational graph.

~~The probabilistic modelling approach enabled by the TreeFlow Python API in conjunction with TensorFlow Probability has similar goals to other phylogenetic probabilistic modelling software. Two notable examples, RevBayes (?) and Blang (?) provide a generic model definition and inference engine which also support phylogenetic models and tree inference. Some notable differences exist between TreeFlow and these which may prove advantageous for certain users. Firstly, RevBayes requires the user to manually specify an MCMC operator scheme, including parameters and weights. This may require substantial user time and expertise to achieve correct and efficient inference. Blang and TreeFlow, on the other hand, are focused on automatic inference methods that require~~

~~less user input. Secondly, TreeFlowand it's Python model specification API are implemented in a widely used computational framework (TensorFlow) and programming language (Python). As a result it can potentially leverage other computational tools implemented in TensorFlow, including less common distributions, machine learning models, differential equation solvers and alternative inference schemes. The automatic differentiation capabilities TensorFlow affords mean TreeFlow can interface with the large and growing collection of models that rely on gradient-based inference. On the other hand, the choice of computational framework incurs a performance cost as evidenced by the results of benchmarking. Blang models can make use of external Java code, but the ecosystem of related software in Java is less developed, and RevBayes does not provide any functionality for interfacing with software that is not implemented in the Rev language~~A promising direction would be to combine TreeFlow's existing model components and gradient computations with a variational distribution over tree topologies, such as the subsplit Bayesian network approach (**?**), enabling joint inference of topology and continuous parameters.

Conclusion

TreeFlow is a software library that provides tools for statistical modelling and inference on phylogenetic problems. Probabilistic graphical modelling provides flexible model definition involving phylogenetic trees and molecular sequences, and automatic differentiation enables the application of modern scalable inference algorithms with a fixed tree topology. TreeFlow's automatic differentiation-based implementations of core gradient computations provide reasonable performance compared to specialized phylogenetic libraries. These building blocks have the potential to be used in valuable phylogenetic analyses of large and complex genetic datasets, and with the next generation of inference methods that can accelerate the inference of tree topologies using gradient information.

Availability

TreeFlow is an open-source Python package. Instructions for installing TreeFlow, and examples of using it as a library and command line application can be found at `https://github.com/christiaanjs/treeflow`. The manual for TreeFlow can be found at `https://treeflow.readthedocs.io`.

Data availability

Datasets, model definitions, starting values, and BEAST 2 XML files used in the biological examples are available at `https://github.com/christiaanjs/treeflow-paper/releases/tag/v0.0.1` or `https://doi.org/10.5061/dryad.v6wwpzh0p`.

Conflicts of Interest

The authors declare that there are no conflicts of interest.